



Client Management Module

Task Brief for Web Developer Candidate



Company: StartupHealer

Module: Admin Portal + Employee/BDA Portal + Client Portal

Prepared for: Candidate Assignment / Technical Interview Task

WE DON'T JUST CONSULT — WE CARE

1. Project Context

StartupHealer is a startup-consulting company that helps early-stage businesses with compliance, registrations, and growth services. Below is what each service means in simple terms, so there is no confusion while building the "Service" field/dropdown in the system:

Service	What It Means (Simple Explanation)
Startup India Certificate	A government recognition certificate for new businesses under the "Startup India" scheme, giving them tax benefits and official startup status.
MSME Certificate	A registration certificate that classifies a business as a Micro, Small, or Medium Enterprise, making it eligible for government schemes and loans.
GST Registration	Registering a business under India's Goods and Services Tax system, required once a business crosses a certain turnover or wants to legally bill customers with tax.
ISO 9001:2015 Certification	A quality-management certification that shows a business follows standard, reliable processes — often required to build trust with bigger clients.
Trademark & Company Registration	Legally registering a business name/brand/logo (trademark) and/or registering the company itself (e.g. Private Limited, LLP) with the government.
Funding & Investment Support	Helping startups prepare for and connect with investors — pitch decks, valuation guidance, and introductions to funding sources.
Web Designing	Building a website for the client's business.
Digital Marketing	Running online marketing (social media, ads, SEO) to help the client's business grow and get customers.

Every day, multiple clients submit applications for one or more of these services, and internal staff (BDAs/Consultants) work on processing them. Currently there is no centralized system to track a client's application status, assign and track daily work for the team, or let clients check their own progress without calling the office. This module solves all three problems with **three connected portals**, all pulling from the same underlying database.

2. Module Scope — Three Portals

A. Admin / Management Portal

Used by leadership/admin to manage clients, assign tasks to employees, set targets, and monitor overall performance.

B. Employee / BDA Portal

Used by internal staff to see daily assigned tasks, update progress by end of day, and track their own targets.

C. Client Self-Service Portal

Used by clients to log in and view only their own profile, services applied for, live status, and documents.

Important: These are three separate login systems with three different roles (Admin, Employee, Client). Each role must only see data relevant to itself — a BDA should not see other BDAs' targets, and a client should never see another client's data or any internal task/target information.

3. Detailed Requirements

3.1 Admin Portal

A. Login Page — Email, Password, Login button, with validation + error messages.

B. Dashboard — Summary cards: Total Clients / Active Clients / Pending Clients / Completed Clients / Total Employees / Tasks Due Today / Tasks Overdue.

C. Client Management — Client list/table, Add / Edit / Delete Client, Search, Status filter, and assign a client to a specific BDA/Employee.

D. Client Fields — Name, Company Name, Email, Phone, Service(s), Status (per service), Date, Assigned Employee.

E. Employee & Task Management — Add/Edit/Remove Employee (Name, Email, Role); assign daily/weekly tasks (title, description, related client, due date); set targets (e.g. "Onboard 10 new clients this month"); view a performance dashboard per employee (tasks completed vs assigned, target vs achieved, EOD history).

F. Auto-Generated Logins — When Admin adds a new Client or Employee, the system auto-generates their portal credentials.

3.2 Employee / BDA Portal

A. Login Page — Email/Employee ID, Password, Login button.

B. Dashboard

- Today's Tasks** — title, description, related client (clickable), due date, status: Not Started In Progress Completed Blocked
- My Targets** — current period, target number, achieved so far, progress bar (e.g. "7/10 clients onboarded")

- **My Assigned Clients** — quick view of status for each
 - **EOD Update** — a simple end-of-day form: what was completed, status update per task, blockers/notes — visible to Admin against that date
- C. Task History** — calendar/list view of past tasks and on-time completion, for self-review and admin performance review.
- D. Salary Slip (New)** — employee can view and download their monthly salary slip(s) as PDF from their portal. Admin uploads/generates the slip against each Employee + Month (a simple upload flow is enough — a full payroll calculation engine is not required for this assignment).
- E. Leave Request & Status (New)** — employee can raise a leave request (Leave Type — Sick/Casual/Paid, From Date, To Date, Reason) and track its status: Pending Approved Rejected. Admin reviews and approves/rejects from the Admin Portal — the status reflects back to the employee instantly from the same shared database.
- F. Attendance System (New)** — employee marks daily attendance (Check-In/Check-Out or "Mark Present") and can view their own attendance history/calendar (Present/Absent/On Leave/Holiday). Admin can view attendance for all employees, individually and as a team summary. A date covered by an approved leave should automatically show as "On Leave" in attendance, not as absent.

3.3 Client Portal

- A. Login Page** — Email/Client ID, Password, Login button (Forgot password can be a placeholder).
- B. Dashboard** — Profile, and all service applications with Service Name, Status, Date Applied, Last Updated, and an Admin/BDA remark field.
- C. Document Upload/Download** — client uploads required documents against a service; once Approved/Completed, final certificate is downloadable from the portal.
- D. Notifications** — simple in-portal alert whenever status changes (a "recent updates" panel is enough; push/SMS/email are optional stretch goals).
- E. Status Update Flow** — only Admin/assigned Employee can change status; updates reflect from the same shared database — no duplicate data source.

3.4 Data Model Note

Design a proper relational schema covering:

- Client** — profile info
- ServiceApplication** — many-to-one to Client, own status/type/dates
- Employee** — profile info, role
- Task** — assigned to one Employee, optionally linked to a Client, due date + status
- Target** — belongs to one Employee, period, target value, achieved value
- EODUpdate** — belongs to one Employee, one per day, free-text + linked task statuses
- SalarySlip** — belongs to one Employee, tied to a month/year, stores the uploaded PDF reference
- LeaveRequest** — belongs to one Employee, leave type, from-date, to-date, reason, status (Pending/Approved/Rejected)
- Attendance** — belongs to one Employee, one record per date, status (Present/Absent/On Leave/Holiday), check-in/check-out time

Relationships: one Employee → many Clients (assigned); one Client → many ServiceApplications; one Employee → many Tasks, EODUpdates, SalarySlips, LeaveRequests, and Attendance records.

3.5 Validation Rules

- Email must be valid format
- Name and Company Name cannot be blank
- Phone number must be validated (10-digit, numeric)
- All required fields enforced on both frontend and backend

3.6 UI/UX Requirements

- Fully responsive — works on mobile, tablet, desktop
- Loading states, error states, empty states (e.g. "No tasks for today", "No clients found")

4. Technology Stack

Frontend: React or Next.js with TypeScript | **Backend:** Spring Boot / FastAPI / Node.js (candidate's choice) | **Database:** candidate's choice, with brief justification.

Strict Rule — No AI Tools for Frontend Development. The frontend code must be written manually by the candidate — no AI code generation tools (ChatGPT, Copilot, Claude, Cursor AI-generated components, v0, Lovable, or similar) are allowed for building the UI/frontend, in any form. This is being evaluated specifically to assess the candidate's own React/Next.js/TypeScript skills, so this rule will be checked carefully during the walkthrough/interview. Backend logic guidance/research is fine, but frontend implementation must be 100% hand-written.

Consequence: If it is found that AI tools were used to generate the frontend code, the submission will be treated as invalid and the candidate's assignment will be rejected. Please follow this rule seriously.

5. UI Theme — Brand Color System

The design must visually match the official StartupHealer logo, so all three portals feel like one consistent product.

				
Startup Blue #1CA3D6	Healer Green #5FAF46	Deep Teal #2E9E8E	Navy Text #1F2937	BG #F7FAFC

Signature gradient (recreate the logo's leaf transition) — use on primary buttons, sidebar accents, dashboard headers:

```
background: linear-gradient(135deg, #5FAF46 0%, #2E9E8E 50%, #1CA3D6 100%);
```

Use this gradient sparingly — on primary CTAs (Login, Add Client, Submit), the navbar/sidebar accent, and dashboard banners. Don't overuse it on every element.

Status tags — color-coded, pill-shaped: Approved / Completed Pending / Under Review Rejected / Blocked In Progress

Design style — clean, modern, card-based (8–12px radius, soft shadows, generous white space); clear typography hierarchy in Navy for headings; rounded buttons (gradient/solid primary, outline secondary); the StartupHealer logo on the navbar/sidebar of every portal; each portal must use the same color system, components, and spacing rules — no visually unrelated apps.

6. Interview Process (Internal Use)

First 5 minutes: ask the candidate to explain their architecture before writing any code.

Key question: "Frontend se database tak data kaise jayega?"

React/Next.js → REST API → Backend (Controller → Service → Repository) → Database

They should explain: how the frontend calls the backend (methods per action); how the backend is layered (Controller → Service → Repository); how the response updates the UI (loading → success/error); and how Admin / Employee / Client auth stay separate (role-based access, route guards). Only after this should implementation begin.

7. Deliverables Expected

Deliverable	Details
Frontend	React/Next.js + TypeScript, covering all three portals
Backend	Working REST APIs
Database	Client, ServiceApplication, Employee, Task, Target, EODUpdate, SalarySlip, LeaveRequest, Attendance — properly related
Authentication	Three working login flows (Admin / Employee / Client) with strict role separation
Documentation	Brief write-up of architecture and folder structure
Demo	Hosted or local demo ready to walk through